

11.LOSSLESS IMAGE COMPRESSION USING HUFFMAN CODING

AIM:

To perform Huffman coding on a grayscale image for lossless image compression, and to reconstruct (decode) the original image from the compressed bitstream to verify data integrity.

INTRODUCTION:

Huffman coding is a widely used entropy encoding algorithm for lossless data compression. It assigns shorter codewords to more probable symbols and longer codewords to less probable ones, thereby minimizing the average code length.

1. Source Symbols and Probabilities

Let the source consist of symbols a_i with probabilities $p(a_i)$, where

$$\sum_i p(a_i) = 1$$

2. Entropy of the Source

The entropy represents the theoretical lower bound on the average number of bits per symbol:

$$H = -\sum_i p(a_i) \log_2(p(a_i))$$

3. Huffman Coding Procedure

1. List the source symbols in order of decreasing probability.
2. Combine the two symbols with the smallest probabilities into a compound symbol, and repeat until only two symbols remain.
3. Assign binary digits (0 and 1) to the final two symbols.
4. Recursively assign codewords by tracing back the merging steps.

This produces a prefix code, meaning no codeword is a prefix of another, ensuring instantaneous and unique decodability.

4. Average Code Length

The average length of a Huffman code is:

$$L_{avg} = \sum_i p(a_i) \cdot l(a_i)$$

where $l(a_i)$ is the length (in bits) of the codeword for symbol a_i .

CODE(MATLAB):

```
close,clc
I=imread("cameraman.tif");
symbols=0:255;
[m,n]=size(I);
p=zeros(1,256);
for i=1:m
    for j=1:n
        x=I(i,j);
        p(x+1)=p(x+1)+1;
    end
end
np=m*n;
prob=p./np;
i2=reshape(I,1,[]);
dic=huffmandict(symbols,prob);
enco=huffmanenco(i2,dic);
deco=huffmandeco(enco,dic);
y=reshape(deco,m,n);
equal=isequal(I,y);
subplot(1,2,1),imshow(I);
subplot(1,2,2),imshow(y,[]);

codeLengths = zeros(1,256);
for k = 1:256
    sym = symbols(k);
    idx = find(cell2mat(dic(:,1)) == sym);
    codeLengths(k) = length(dic{idx,2});
end
Lavg = sum(prob .* codeLengths);
entropy = -sum(prob(prob>0).*log2(prob(prob>0)));
bits = 8;
CR = bits / Lavg;
R=1-(1/CR);
fprintf('Entropy: %.2f bits\n', entropy);
fprintf('Average Code Length: %.2f bits\n', Lavg);
fprintf('Redundancy: %.2f bits\n', R);
fprintf('Compression ratio: %.2f\n', CR);
```

Output:



```
Entropy: 7.01 bits  
Average Code Length: 7.04 bits  
Redundancy: 0.12 bits  
Compression ratio: 1.14
```

Figure: (a) The original input grayscale image before compression. (b) The same image after applying Huffman encoding, where redundancy is reduced while preserving visual quality.

INFERENCE:

Huffman coding successfully reduced the number of bits required to represent the grayscale image compared to the original fixed-length representation.

The reconstructed (decoded) image was identical to the original, confirming that Huffman coding is lossless.

CONCLUSION:

Huffman coding is an effective lossless image compression technique that reduces storage and transmission requirements without losing any information.

Overall, Huffman coding demonstrates the importance of probability-based encoding for efficient data representation in digital images.